TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Secure Code Debugging and Review Assistant

Student Details

Field	Details
Institute/University	PCCOE (confirm official name)
Programme / Batch	Applied AI/ML Capstone — TechPulse FY-26
Student Name	_____
PRN	_____
Case Study ID (CS1–CS5)	Case Study 4 (confirm official ID)
Project Title / Topic	Secure Code Debugging and Review Assistant

1. Case Study ID and Project Title / Topic

Case Study ID: Case Study 4 (confirm the official CS1–CS5 mapping with faculty).
Project title: Secure Code Debugging and Review Assistant.

2. Objectives

• Develop a Streamlit interface for submitting Python source code for review.
• Use Bandit and Ruff to identify potential security and code-quality issues.
• Retrieve relevant secure-coding guidance from a local knowledge base using Sentence Transformers and ChromaDB.
• Present findings with severity, line numbers, evidence, and relevant guideline passages.
• Generate a downloadable Markdown report and evaluate the prototype using vulnerable, clean, and malformed code samples.

3. Scope

In Scope – To Be Implemented
• Python source-code review using available static-analysis tools.
• Local indexing and retrieval of approved or synthetic secure-coding guidelines.
• Display of findings and guideline evidence through Streamlit.
• Markdown report export and evaluation using a small test-case set.

Out of Scope – Will Not Be Implemented
• Guaranteed detection of all vulnerabilities or replacement of expert security audits.
• Automatic execution of submitted code or automatic application of fixes.
• Review of confidential/proprietary code through an external API without authorization.
• Claims of measured precision/recall until a larger labelled ground-truth dataset is evaluated.

4. Proposed Approach

Component	Proposed / current choice
Pre-trained model	Optional OpenAI model via API; code default is gpt-4.1-mini. Not configured in the current tested run.
Embedding model	sentence-transformers/all-MiniLM-L6-v2.
Vector store	ChromaDB PersistentClient; local chroma_db directory; collection coding_guidelines.
RAG pipeline / AI-agent design	Finding-aware retrieval queries search local secure-coding guideline chunks. This is a RAG workflow, not a multi-agent system.
Service layer and UI	Python and Streamlit; main UI in app.py.
Tools / deterministic rules	Bandit and Ruff for static analysis; section-aware chunking; ChromaDB retrieval; Markdown report generation.

Proposed System / Architecture

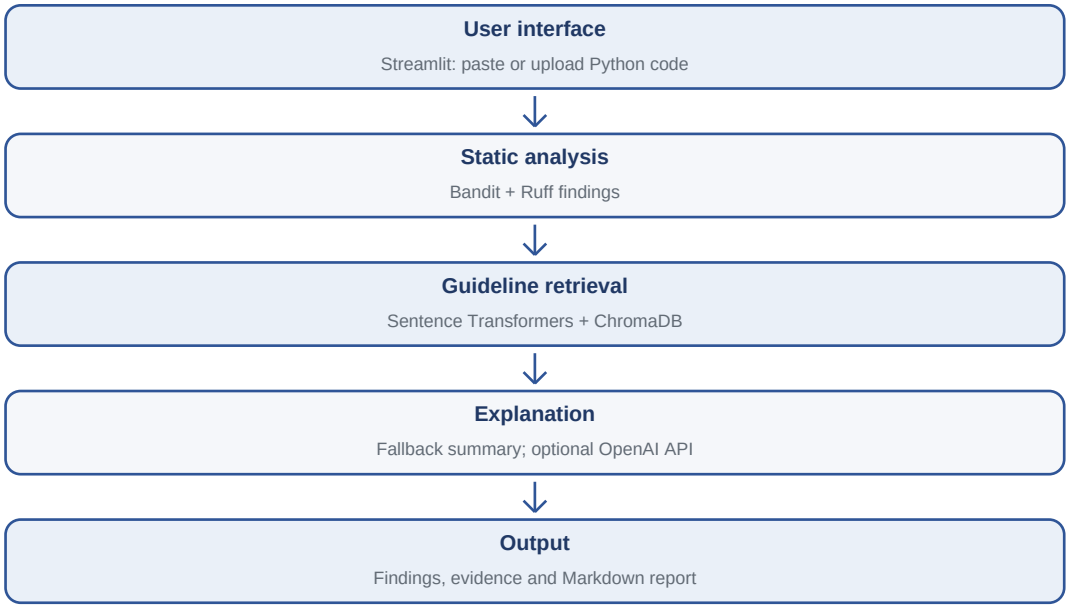

Figure 1. Proposed workflow of the Secure Code Review Assistant.

5. Input Data / Knowledge Base

Item	Description
Data / Knowledge Sources	Local Markdown/TXT guideline files under data/guidelines/; current demo file coding_guidelines_demo.md. Use approved or synthetic examples only.
Approximate Size and Format	Small local Markdown document; the observed index contained 11 guideline chunks after section-aware indexing.
Preprocessing and EDA Planned	Read text as UTF-8, separate sections using Markdown headings, split long sections into chunks of up to 180 words, and retain source/chunk metadata. Re-index after guideline changes.

6. Expected Outcomes and Evaluation Plan

Expected Outputs

- Findings for suspicious code patterns, including severity, rule ID, line number, and evidence.
- Relevant secure-coding guideline passages with source and chunk labels.
- Downloadable Markdown report and summary of limitations.
- A documented evaluation set with vulnerable, clean, syntax-error, safer-subprocess, and input-validation scenarios.

Evaluation Method

Use the six recorded cases in evaluation/test_cases.csv. Compare actual analyzer output with expected behavior; retain screenshots and exported reports. The current CSV marks 6/6 cases as Pass, but this is a test-case pass rate, not measured detection accuracy. TC01 and TC02 overlap. Retrieval relevance is inspected qualitatively; formal precision, recall, latency, and

retrieval hit rate remain to be measured.

Responsible AI Measures
Retrieved evidence is shown with source/chunk information. Guideline retrieval and the vector index are local. The optional OpenAI API may transmit submitted code to an external provider; confidential code must not be submitted without authorization. Findings and explanations are advisory and require human review and testing.

MLOps / Deployment Elements Planned
Version source code and evaluation data with Git; maintain requirements.txt and documented setup steps. Optional future improvements include Docker packaging and automated tests/CI. These deployment improvements are planned, not claimed as completed.

7. Planned AI-Tool Usage

AI Tool / Model	Purpose	Artifact Affected
ChatGPT / OpenAI assistant	Assistance with debugging, explanations, drafting, and report structure; student must verify all generated material.	Code changes and project documentation; disclose accurately in final declaration.
Sentence Transformers: all-MiniLM-L6-v2	Create embeddings for guideline chunks and retrieval queries.	Local RAG pipeline; tested during retrieval.
OpenAI API model (optional)	Generate natural-language review explanations when API configuration is enabled.	Optional explanation function; not configured in current test run.

Student Declaration

■ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

■ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature: _____ Date: _____

Before submission: Fill in student details, verify the official case-study ID and institute name, and review all descriptions against your final implementation. The optional LLM is marked as unconfigured, and deployment enhancements are marked as planned.